

ONEDRAFT

CAD-AI — Aria Powered

Data Lifecycle, Retention & Provenance System

Version 1.0

Status: Implementation Specification

Database: PostgreSQL

Artifact Storage: Object Storage

API: REST / JSON

API Version: /api/v1

Identifiers: UUID

Primary Model: Versioned OneDraft Project Model

AI Layer: Aria Powered

1. DATA LIFECYCLE DESIGN PRINCIPLE

OneDraft shall treat project data as a controlled lifecycle rather than as undifferentiated application storage.

Every significant project artifact, model state, regulatory source, validation result, drawing, document and acceptance record shall possess identifiable provenance.

The system shall distinguish:

Active Data

Current operational project information.

Historical Data

Previous project, model, revision and configuration states.

Derived Data

Geometry, drawings, schedules, validation outputs and document packages generated from authoritative project state.

Immutable Evidence

Hashes, audit events, acceptance records, regulatory manifests and other records whose historical integrity must be preserved.

Archived Data

Data retained for historical, contractual, regulatory or operational purposes but removed from active working paths.

Deleted Data

Data that has passed its applicable retention requirements and is authorized for destruction.

The lifecycle system therefore governs:

```
CREATE
↓
ACTIVE
↓
VERSIONED
↓
HISTORICAL
↓
ARCHIVED
↓
RETENTION EXPIRY
↓
AUTHORIZED DELETION
```

2. DATA OWNERSHIP PRINCIPLE

Every persistent OneDraft object shall have an identifiable ownership context.

The minimum ownership hierarchy is:

```
PLATFORM
↓
ORGANIZATION
↓
PROJECT
↓
OBJECT
↓
REVISION
↓
DERIVED ARTIFACT
```

Ownership determines:

access

retention

archival

deletion authority

provenance

audit requirements

No artifact may become orphaned from its project ownership context.

3. DATA CLASSIFICATION

OneDraft shall classify persistent data according to operational significance.

Initial classifications:

PROJECT_DATA
MODEL_DATA
GEOMETRY_DATA
COMPLIANCE_DATA
VALIDATION_DATA
DOCUMENTATION_DATA
AUDIT_DATA
IDENTITY_DATA
BILLING_DATA
SYSTEM_DATA
ARTIFACT_DATA

Classification shall be stored through explicit metadata where required.

4. DATA SENSITIVITY

The lifecycle system shall support sensitivity classifications.

Initial values:

PUBLIC
INTERNAL
CONFIDENTIAL
RESTRICTED
SYSTEM

Sensitivity classification does not replace authorization.

Authorization remains governed by the Identity, Access Control & Security System.

5. DATA RECORD METADATA

Persistent records should support lifecycle metadata where applicable:

id
created_at
updated_at
created_by
updated_by

organization_id
project_id
status
data_classification
retention_policy_id
archived_at
deleted_at

Not every table requires every field.

The implementation shall avoid unnecessary duplication where ownership can be derived through relational relationships.

6. LIFECYCLE STATES

Standard lifecycle states shall include:

DRAFT
ACTIVE
SUPERSEDED
HISTORICAL
ARCHIVED
LOCKED
PENDING_DELETION
DELETED

Certain subsystems may define additional domain-specific states.

Lifecycle state transitions must be controlled by application services rather than unrestricted database mutation.

7. STATE TRANSITION RULE

A lifecycle transition shall be represented conceptually as:

```
graph TD
    A[CURRENT STATE] --> B[VALIDATE TRANSITION]
    B --> C[AUTHORIZE]
    C --> D[RECORD EVENT]
    D --> E[UPDATE STATE]
    E --> F[COMMIT]
```

Invalid transitions shall be rejected.

For example:

DELETED → ACTIVE

shall not be permitted through ordinary application operations.

8. PROJECT DATA LIFECYCLE

A project shall progress through:

INTAKE
↓
SPECIFICATION
↓
MODELING
↓
VALIDATION
↓
DOCUMENTATION
↓
REVIEW
↓
ACCEPTANCE
↓
FINALIZED
↓
ARCHIVED

The lifecycle system shall preserve the relationship between project state and the underlying model/revision state.

9. MODEL DATA LIFECYCLE

The computational project model shall be versioned.

Conceptually:

MODEL VERSION 1
↓
MODEL VERSION 2
↓
MODEL VERSION 3
↓
MODEL VERSION N

A newer model version does not destroy the previous version.

Previous versions become historical project state.

10. REVISION RETENTION

Every finalized revision shall remain reconstructable for the applicable retention period.

A revision shall retain references to:

- project_id
- model_version_id
- parent_revision_id
- code_manifest_id
- validation_results
- drawing_set
- document_package
- change_records
- acceptance_records

This allows the complete revision state to be reconstructed.

11. DERIVED DATA PROVENANCE

Derived artifacts shall identify the authoritative state from which they were generated.

A drawing shall identify:

- project_id
- revision_id
- model_version_id
- view_id

A validation result shall identify:

- project_id
- revision_id
- model_version_id
- code_manifest_id
- validation_run_id

A document package shall identify:

- project_id
- revision_id
- model_version_id
- code_manifest_id
- validation_run_id

12. PROVENANCE CHAIN

The minimum provenance chain shall be:

PROJECT
↓
MODEL VERSION
↓
REVISION
↓
CODE MANIFEST
↓
VALIDATION RUN
↓
DRAWING SET
↓
DOCUMENT PACKAGE
↓
ACCEPTANCE

Each derived stage shall be traceable backward to its authoritative source.

13. ARTIFACT PROVENANCE

Every stored artifact shall maintain provenance metadata.

Conceptually:

artifact_id
project_id
revision_id
model_version_id
artifact_type
source_system
generation_job_id
content_hash
created_at
created_by

Artifacts lacking sufficient provenance shall not be classified as final project deliverables.

14. CONTENT HASHING

Important persistent artifacts shall receive cryptographic content hashes.

The initial implementation shall support:

SHA-256

The hash shall represent the stored artifact content.

Example:

document.pdf
↓

SHA-256

↓

document_hash

The hash provides an integrity identifier rather than a substitute for access control.

15. MODEL STATE HASHING

Model versions shall contain a deterministic state hash.

Conceptually:

MODEL OBJECTS

+

RELATIONSHIPS

+

CONSTRAINTS

+

PARAMETERS

↓

CANONICAL SERIALIZATION

↓

HASH

↓

STATE_HASH

The canonicalization process must be deterministic.

Equivalent model states must produce the same canonical representation before hashing.

16. REVISION HASHING

Where required, a revision may additionally maintain a deterministic revision hash.

The revision hash shall incorporate the authoritative model state and revision metadata necessary to identify that revision.

This allows later verification that a reconstructed revision corresponds to the originally recorded state.

17. ARTIFACT STORAGE

Large binary artifacts shall not normally be stored directly inside PostgreSQL.

The architecture shall use object storage for:

PDF

DWG

DXF
IFC
SVG
PNG
JPEG
ZIP
CAD exports
generated document packages
source documents
large geometry payloads

PostgreSQL stores metadata and object-storage references.

18. ARTIFACT REFERENCE

The artifact record shall conceptually contain:

```
id UUID
project_id UUID
revision_id UUID
artifact_type VARCHAR(64)
storage_provider VARCHAR(64)
storage_bucket VARCHAR(255)
storage_key TEXT
content_hash VARCHAR(128)
content_size BIGINT
mime_type VARCHAR(255)
created_at TIMESTAMPTZ
```

The exact physical implementation may vary by deployment environment.

19. OBJECT STORAGE INTEGRITY

An artifact shall not be considered successfully stored until:

```
OBJECT UPLOAD
↓
OBJECT VERIFICATION
↓
HASH VERIFICATION
↓
DATABASE REFERENCE
↓
COMMIT
```

has completed successfully.

A database reference to a nonexistent or unverified artifact shall not become authoritative.

20. SOURCE DOCUMENTS

Imported source documents shall preserve their original identity and provenance.

Examples:

SURVEY
CODE DOCUMENT
OWNER SPECIFICATION
CONSULTANT DOCUMENT
EXISTING CAD
BIM MODEL
REFERENCE IMAGE
PRODUCT DATA
PROJECT BRIEF

The original source artifact shall remain distinct from any parsed or transformed representation.

21. SOURCE DOCUMENT HASH

Every imported source document should receive:

content_hash
content_size
mime_type
source_reference
import_timestamp
imported_by

If the same source document is imported repeatedly, the content hash can be used to detect duplicates.

22. PARSED DATA PROVENANCE

When information is extracted from a source document, the resulting structured data shall retain a reference to the source.

Conceptually:

SOURCE DOCUMENT
↓
EXTRACTION
↓
STRUCTURED PROJECT DATA

The structured data shall identify the source artifact where practical.

This prevents extracted information from becoming detached from its originating document.

23. AI-GENERATED DATA PROVENANCE

AI-generated project modifications shall identify the originating AI operation.

Conceptually:

```
USER REQUEST
↓
ARIA COMMAND
↓
MODEL OPERATION
↓
REVISION
```

The resulting revision shall therefore be traceable to the originating command.

Where applicable, the system shall retain:

```
command_id
request_id
model/provider metadata
command parameters
validation requirements
execution result
```

Provider-specific information shall remain behind the AI provider abstraction.

24. HUMAN-GENERATED DATA PROVENANCE

Human-authored changes shall also be recorded.

Examples include:

```
MANUAL EDIT
REDLINE
APPROVAL
REJECTION
ACCEPTANCE
COMMENT
REVIEW
OVERRIDE
```

The system shall distinguish the originating actor from the operation itself.

25. MACHINE-GENERATED DATA PROVENANCE

Automated workers shall produce identifiable execution metadata.

Examples:

```
worker_id
job_id
execution_id
service_name
software_version
created_at
```

This permits a generated artifact to be traced to the execution environment that produced it.

26. PROVENANCE GRAPH

OneDraft shall conceptually maintain a provenance graph:

```
SOURCE
↓
REQUIREMENT
↓
MODEL
↓
COMMAND
↓
REVISION
↓
VALIDATION
↓
DRAWING
↓
DOCUMENT
↓
ACCEPTANCE
```

The graph may be represented through existing relational references rather than requiring a dedicated graph database in the MVP.

27. REQUIREMENT PROVENANCE

Each project requirement shall identify its originating source.

Possible sources:

```
CUSTOMER
```

JURISDICTION
CODE
CONSULTANT
PROJECT_STANDARD
ARIA
SYSTEM

Requirements shall remain traceable to the source that created them.

28. COMPLIANCE PROVENANCE

Regulatory validation shall identify the code source and manifest used.

The provenance chain shall include:

```
CODE SOURCE
↓
CODE RULE
↓
CODE MANIFEST
↓
VALIDATION RUN
↓
VALIDATION RESULT
```

A finalized revision shall therefore preserve the regulatory basis used during validation.

29. IMMUTABLE REGULATORY MANIFESTS

Once a code manifest has been attached to a finalized revision, it shall become immutable.

If the applicable regulatory source changes, OneDraft shall create a new manifest rather than modifying the previous one.

Conceptually:

```
MANIFEST 1
↓
FINALIZED REVISION

MANIFEST 2
↓
NEW REVISION
```

Historical regulatory context must remain reconstructable.

30. VALIDATION PROVENANCE

Every validation result shall identify:

validation_run_id
revision_id
model_version_id
code_manifest_id
rule_id
affected_objects
execution_timestamp

This establishes which project state was evaluated against which regulatory rules.

31. DOCUMENT PROVENANCE

A final document package shall never exist solely as an unqualified file.

The package must be associated with:

PROJECT
REVISION
MODEL VERSION
CODE MANIFEST
VALIDATION RUN
GENERATION JOB
DOCUMENT HASH

This makes the document package reproducible and auditable.

32. ACCEPTANCE PROVENANCE

Customer acceptance shall identify:

customer_id
project_id
revision_id
model_version_id
document_package_id
accepted_at
acceptance_status

Acceptance therefore applies to a defined project state rather than to an unspecified collection of files.

33. AUDIT PROVENANCE

All material lifecycle transitions shall produce audit events.

Examples:

PROJECT_CREATED
MODEL_VERSION_CREATED
REVISION_CREATED
COMMAND_EXECUTED
VALIDATION_COMPLETED
DRAWING_GENERATED
DOCUMENT_GENERATED
ARTIFACT_STORED
ARTIFACT_ARCHIVED
ACCEPTANCE_RECORDED
DATA_DELETED

The audit event shall identify the actor and affected project/object where applicable.

34. RETENTION POLICY

Retention shall be policy-driven.

A retention policy shall conceptually define:

policy_id
data_classification
retention_period
archive_after
deletion_after
legal_hold_behavior
status

Retention periods shall not be hard-coded throughout application services.

35. RETENTION POLICY ASSOCIATION

Data objects shall resolve their retention policy through classification and ownership.

Conceptually:

OBJECT
↓
DATA CLASSIFICATION
↓
RETENTION POLICY
↓
LIFECYCLE ACTION

This permits retention requirements to evolve without rewriting every storage subsystem.

36. RETENTION CLOCK

Retention timing shall be based on an explicit lifecycle event.

Possible anchors include:

CREATION
FINALIZATION
ACCEPTANCE
PROJECT CLOSURE
LAST ACTIVITY
ARCHIVAL

The applicable anchor must be defined by the relevant retention policy.

37. ARCHIVAL

Archival shall move data from active operational storage into controlled historical storage.

Conceptually:

ACTIVE DATA
↓
ARCHIVE ELIGIBILITY
↓
ARCHIVAL JOB
↓
INTEGRITY CHECK
↓
ARCHIVED

Archival shall preserve provenance.

38. ARCHIVE MANIFEST

An archival operation should create an archive manifest containing:

archive_id
project_id
objects
artifact_references
content_hashes
source_versions
created_at
archived_at

archive_format

The manifest permits later verification of the archived dataset.

39. ARCHIVE INTEGRITY

Archived data shall remain verifiable.

The system should periodically verify:

stored object exists
content hash matches
metadata remains consistent
provenance references remain resolvable

Failures shall generate operational alerts.

40. PROJECT ARCHIVAL

A project may become eligible for archival after completion of its active lifecycle.

Conceptually:

ACCEPTED
↓
FINALIZED
↓
INACTIVE
↓
ARCHIVE ELIGIBLE
↓
ARCHIVED

Archival shall not destroy the project's historical state.

41. LEGAL OR ADMINISTRATIVE HOLD

The lifecycle system shall support a hold mechanism.

A held object shall not be automatically deleted while the hold remains active.

Conceptually:

RETENTION EXPIRY
+
ACTIVE HOLD
=

NO DELETION

A hold should identify:

hold_id
scope
reason
created_by
created_at
released_by
released_at

42. DELETION ELIGIBILITY

An object becomes deletion-eligible only when:

RETENTION EXPIRED
AND
NO ACTIVE HOLD
AND
NO REQUIRED DEPENDENCY
AND
AUTHORIZED DELETION POLICY EXISTS

Eligibility does not itself execute deletion.

43. DELETION AUTHORIZATION

Deletion shall pass through an explicit application operation.

Conceptually:

DELETION CANDIDATE
↓
POLICY CHECK
↓
DEPENDENCY CHECK
↓
HOLD CHECK
↓
AUTHORIZATION
↓
DELETION JOB

The operation shall be auditable.

44. SOFT DELETION

User-facing objects should initially use soft deletion.

Conceptually:

```
status = DELETED  
deleted_at = timestamp
```

The object remains available for controlled recovery or historical reference until permanent deletion is authorized.

45. HARD DELETION

Permanent deletion shall be reserved for data that has satisfied all applicable deletion conditions.

Hard deletion may include:

```
database record removal  
object-storage deletion  
derived-cache deletion  
search-index removal  
secondary replica expiration
```

The system must account for all storage locations.

46. CASCADE DELETION CONTROL

OneDraft shall avoid uncontrolled database cascade deletion for historically significant project records.

Relationships involving:

```
revisions  
model_versions  
validation_results  
document_packages  
acceptance_records  
audit_events
```

shall receive explicit lifecycle treatment.

The objective is to prevent accidental destruction of provenance.

47. DEPENDENCY CHECKING

Before deleting an object, the system shall determine whether other authoritative or historical objects depend upon it.

Example:

```
MODEL VERSION
↓
REVISION
↓
DRAWING
↓
DOCUMENT PACKAGE
↓
ACCEPTANCE
```

A model version supporting a finalized document package shall not be silently deleted.

48. DERIVED DATA REGENERATION

Where a derived artifact can be regenerated from authoritative state, the system may prioritize preservation of the authoritative source over indefinite retention of every generated derivative.

For example:

```
PROJECT MODEL
+
VIEW
+
DRAWING RULES
↓
DRAWING
```

However, finalized deliverables may still require independent retention according to contractual or regulatory policy.

49. CACHE DATA

Temporary computational data may have shorter retention.

Examples:

```
geometry cache
render cache
temporary previews
worker scratch files
temporary exports
job intermediates
```

These records may be automatically expired after successful completion unless required for debugging, audit or reproducibility.

50. JOB ARTIFACT RETENTION

Worker jobs shall distinguish:

temporary artifact
intermediate artifact
final artifact
failed-job artifact
diagnostic artifact

Temporary artifacts may be automatically removed after a defined period.

Final artifacts remain governed by project retention policy.

51. EVENT RETENTION

Event records shall receive explicit retention classification.

Material project events should remain available for the applicable historical period.

Operational telemetry events may use shorter retention.

This distinction prevents high-volume infrastructure telemetry from being confused with authoritative project audit history.

52. AUDIT EVENT IMMUTABILITY

Authoritative audit events shall be append-only.

Existing events shall not normally be updated.

Corrections shall be represented by additional events.

Conceptually:

ORIGINAL EVENT
↓
CORRECTION EVENT

rather than:

UPDATE ORIGINAL EVENT

53. PROVENANCE IMMUTABILITY

The following records shall become immutable when finalized:

FINALIZED MODEL VERSION
FINALIZED REVISION
FINALIZED CODE MANIFEST
FINAL VALIDATION RUN
FINAL DOCUMENT PACKAGE
CUSTOMER ACCEPTANCE

Any subsequent change produces a new version or revision.

54. FINALIZATION BOUNDARY

Finalization shall create a controlled boundary between mutable working data and historical project state.

Conceptually:

WORKING MODEL
↓
VALIDATION
↓
REVIEW
↓
FINALIZATION
↓
IMMUTABLE PROJECT STATE

The finalization operation shall be recorded in the audit system.

55. RECONSTRUCTION REQUIREMENT

The lifecycle architecture shall support reconstruction of a historical project state.

Given:

project_id
revision_id
model_version_id
code_manifest_id

the system must be able to identify:

authoritative model state

applicable requirements
applicable constraints
regulatory basis
validation results
drawing references
document package
acceptance state

56. PROVENANCE VERIFICATION

OneDraft shall provide an internal verification process.

Conceptually:

```
SELECT HISTORICAL REVISION
↓
RESOLVE MODEL
↓
RESOLVE CODE MANIFEST
↓
RESOLVE VALIDATION
↓
RESOLVE DRAWINGS
↓
RESOLVE DOCUMENT PACKAGE
↓
VERIFY HASHES
↓
REPORT RESULT
```

The verification result shall identify any missing or inconsistent provenance.

57. DATA INTEGRITY CHECK

The lifecycle system shall periodically check for:

orphaned records
missing artifacts
hash mismatches
broken references
invalid lifecycle states
invalid ownership
missing provenance
inconsistent revision references

Integrity failures shall be surfaced through the Observability and System Health subsystem.

58. DATA LINEAGE

Lineage shall describe how information moved through the system.

Example:

```
OWNER SPECIFICATION
↓
PROJECT REQUIREMENT
↓
ARIA COMMAND
↓
MODEL CHANGE
↓
REVISION
↓
DRAWING
↓
DOCUMENT
```

Lineage is distinct from audit history.

Audit answers:

What happened?

Lineage answers:

Where did this information come from?

59. SOURCE-TO-OUTPUT TRACEABILITY

OneDraft should eventually support a traceability query such as:

Which project requirement caused this drawing annotation?

The system shall traverse:

```
REQUIREMENT
↓
AFFECTED OBJECT
↓
REVISION
↓
VIEW
↓
ANNOTATION
```

This becomes important for complex projects and regulatory review.

60. OUTPUT-TO-SOURCE TRACEABILITY

The reverse query shall also be supported:

Which drawings and documents were affected by this requirement?

The dependency graph shall identify downstream outputs.

61. DATA LINEAGE API

The future API shall support a provenance endpoint conceptually similar to:

GET /api/v1/projects/{project_id}/provenance

Possible query parameters:

revision_id
object_id
artifact_id
direction
depth

Possible directions:

UPSTREAM
DOWNSTREAM
BOTH

62. ARTIFACT HISTORY API

The API shall support historical artifact inspection.

Conceptually:

GET /api/v1/projects/{project_id}/artifacts

Filters may include:

artifact_type
revision_id
created_after
created_before
status
hash

63. RETENTION MANAGEMENT API

Administrative services may expose:

```
GET /api/v1/admin/retention/policies
GET /api/v1/admin/retention/candidates
POST /api/v1/admin/retention/archive
POST /api/v1/admin/retention/delete
```

These operations require administrative authorization.

64. DATA LIFECYCLE JOBS

Long-running lifecycle operations shall use the Job & Worker Execution System.

Examples:

```
ARCHIVE_PROJECT
VERIFY_ARCHIVE
DELETE_EXPIRED_DATA
VERIFY_ARTIFACT_INTEGRITY
REBUILD_PROVENANCE
RECONSTRUCT_REVISION
```

The lifecycle system shall therefore integrate with the existing worker architecture rather than creating an independent job system.

65. EVENT BUS INTEGRATION

Lifecycle events shall be published through the Integration & Event Bus.

Examples:

```
artifact.created
artifact.archived
artifact.deleted
revision.finalized
project.archived
retention.expired
deletion.completed
provenance.verified
integrity.failure
```

Consumers may include:

```
audit service
observability service
notification service
administrative control plane
```

66. BACKUP INTEGRATION

The lifecycle system shall cooperate with the Backup, Disaster Recovery & Business Continuity subsystem.

Retention and backup are distinct controls.

Deletion from active storage does not automatically imply immediate destruction from every backup copy.

Backup expiration shall therefore follow the separate backup retention policy.

67. BACKUP / DELETION DISTINCTION

The architecture shall distinguish:

APPLICATION DELETION

from:

BACKUP EXPIRATION

An object deleted from the active system may remain temporarily within retained backups according to the applicable recovery policy.

68. MULTI-TENANT DATA LIFECYCLE

All lifecycle operations must respect organization boundaries.

Conceptually:

ORGANIZATION

↓

PROJECT

↓

DATA

An administrator for one organization must never be able to archive, export or delete another organization's project data through ordinary tenant-scoped operations.

69. PROJECT ISOLATION

Lifecycle jobs shall always carry sufficient ownership context.

Minimum context:

```
organization_id  
project_id
```

Workers must verify this context before executing destructive or archival operations.

70. DATA EXPORT

A project export shall preserve provenance.

A complete export may contain:

```
project metadata  
requirements  
model versions  
revisions  
geometry references  
compliance manifests  
validation results  
drawings  
schedules  
document packages  
audit records  
provenance manifest
```

The export should include a machine-readable manifest.

71. EXPORT MANIFEST

Conceptually:

```
{  
  "project_id": "uuid",  
  "export_id": "uuid",  
  "exported_at": "ISO-8601",  
  "revision_ids": [],  
  "artifact_ids": [],  
  "content_hashes": {},  
  "schema_version": "0.1"  
}
```

The manifest permits verification of the exported dataset.

72. IMPORT PROVENANCE

Imported OneDraft project data shall not overwrite an existing project state without an explicit import operation.

The import process shall create an identifiable event:

```
PROJECT_IMPORT_STARTED  
PROJECT_IMPORT_VALIDATED  
PROJECT_IMPORT_COMPLETED
```

Imported objects shall retain their source provenance where possible.

73. SCHEMA VERSION PROVENANCE

Persistent exports and serialized project data shall identify the OneDraft schema version used to produce them.

Example:

```
schema_version = 0.1
```

Future migrations shall provide controlled conversion paths.

74. MIGRATION PROVENANCE

Database migrations shall be immutable and versioned.

Each deployment shall identify:

```
migration_version  
migration_timestamp  
application_version
```

The operational environment must therefore be able to determine which schema version produced a given project state.

75. SOFTWARE VERSION PROVENANCE

Finalized generated artifacts should, where appropriate, retain the software release that produced them.

Conceptually:

```
application_version  
geometry_engine_version
```

documentation_engine_version
compliance_engine_version
aria_orchestration_version

This improves reproducibility of historical outputs.

76. DETERMINISTIC REGENERATION

Where deterministic regeneration is supported, the system should be able to reproduce a derived artifact from:

project state
revision
software version
configuration
template
code manifest
generation parameters

The resulting artifact can then be compared through content hashes.

77. NON-DETERMINISTIC AI OUTPUT

AI-generated operations may contain non-deterministic model behavior.

Therefore, OneDraft shall preserve the resulting command and resulting project state rather than relying solely on the ability to regenerate the exact AI response.

The authoritative record is the executed and validated project mutation.

78. PROVENANCE OF AI COMMANDS

An AI command record shall preserve:

command_id
project_id
revision_id
request_id
operation
target_objects
parameters
preserve_constraints
validation_requirements
result
status
created_at

completed_at

This record connects AI reasoning output to deterministic application execution.

79. DATA CONSISTENCY RULE

No derived artifact shall become authoritative merely because it exists in storage.

Authority follows the project model and version chain.

Conceptually:

```
MODEL STATE
  ↓
REVISION
  ↓
VALIDATED DERIVATION
  ↓
DOCUMENTATION
```

A file outside this chain is an external or unverified artifact.

80. ORPHAN ARTIFACT DETECTION

The system shall periodically identify artifacts that lack valid references to:

```
project
revision
model_version
generation job
```

Orphan artifacts may be:

```
quarantined
re-associated
archived
deleted
```

according to policy.

81. QUARANTINE

Potentially corrupt or untrusted artifacts may enter:

QUARANTINED

state.

A quarantined artifact shall not be presented as a final project deliverable.

Possible causes include:

hash mismatch
malformed file
unknown provenance
failed import
failed validation
security scan failure

82. SECURITY SCAN INTEGRATION

Imported artifacts may pass through the security subsystem before becoming active project data.

Conceptually:

UPLOAD
↓
QUARANTINE
↓
SECURITY CHECK
↓
HASH
↓
PROVENANCE
↓
ACTIVE

The security system remains responsible for the actual scanning policy.

83. DATA QUALITY STATES

Data may additionally carry quality state:

UNVERIFIED
VERIFIED
VALIDATED
FINAL
SUPERSEDED
QUARANTINED

Quality state and lifecycle state shall remain separate concepts.

84. DATA RETENTION METRICS

The system shall expose operational metrics including:

```
active_project_count
archived_project_count
artifact_count
artifact_storage_size
retention_candidates
deletion_candidates
quarantined_artifacts
orphan_artifacts
hash_verification_failures
provenance_failures
```

These metrics integrate with the Observability subsystem.

85. STORAGE OPTIMIZATION

Storage optimization may include:

```
deduplication
compression
tiered storage
artifact lifecycle policies
cache expiration
archive storage
```

Optimization shall never compromise authoritative project provenance.

86. DUPLICATE DETECTION

Content hashes may identify duplicate artifacts.

For example:

```
HASH A
↓
Artifact 1
Artifact 2
Artifact 3
```

The system may store a single physical object with multiple logical references where appropriate.

Logical ownership and provenance references must remain distinct.

87. ARTIFACT VERSIONING

Artifacts may be versioned independently from project revisions when required.

However, a final project artifact must always identify the project revision to which it belongs.

Example:

```
DRAWING ARTIFACT
↓
Drawing Version 4
↓
Revision 12
```

88. DOCUMENT PACKAGE IMMUTABILITY

Once a document package has been accepted or finalized, its contents shall not be modified in place.

A change creates:

```
NEW REVISION
+
NEW DOCUMENT PACKAGE
```

The previous package remains historically identifiable.

89. ACCEPTANCE LOCK

A customer acceptance record shall establish an immutable reference to the accepted package.

Conceptually:

```
DOCUMENT PACKAGE
↓
CUSTOMER ACCEPTANCE
↓
ACCEPTED PROJECT STATE
```

Subsequent changes must not mutate the accepted package.

90. RETENTION AUDIT

Administrative retention operations shall themselves be audited.

Examples:

RETENTION_POLICY_CREATED
RETENTION_POLICY_CHANGED
ARCHIVE_REQUESTED
ARCHIVE_COMPLETED
DELETE_REQUESTED
DELETE_COMPLETED
HOLD_CREATED
HOLD_RELEASED

This ensures lifecycle management is itself traceable.

91. ADMINISTRATIVE OVERRIDE

Exceptional lifecycle operations may require administrative override.

Any override must record:

actor
reason
scope
previous_state
new_state
timestamp
authorization_context

Overrides shall not silently bypass audit controls.

92. DATA LIFECYCLE API BOUNDARY

External clients shall not directly manipulate lifecycle metadata through arbitrary database fields.

The API shall expose controlled operations.

Examples:

archive project
restore eligible project
create hold
release hold
request export
verify provenance
inspect retention status

93. DATABASE RESPONSIBILITY

PostgreSQL remains responsible for:

metadata
relationships
ownership
lifecycle state
retention references
provenance references
hash metadata
audit relationships

Object storage remains responsible for large binary payloads.

94. OBJECT STORAGE RESPONSIBILITY

Object storage is responsible for:

large artifacts
generated documents
CAD files
BIM files
source documents
rendered drawings
archives

The object-storage layer shall not become the authoritative source for relational project state.

95. CACHE RESPONSIBILITY

Caches remain non-authoritative.

A cache may be safely discarded and rebuilt from authoritative state where regeneration is supported.

Conceptually:

AUTHORITATIVE STATE
↓
CACHE

never:

CACHE
↓
AUTHORITATIVE STATE

96. PROVENANCE SERVICE

OneDraft should expose an internal provenance service responsible for:

- resolve lineage
- verify references
- verify hashes
- construct provenance graph
- identify source objects
- identify derived outputs

This service becomes a common capability for review, audit and export.

97. DATA LIFECYCLE SERVICE

The lifecycle service shall provide controlled operations for:

- classify
- retain
- archive
- restore
- hold
- release
- delete
- verify

The service shall coordinate with:

- Project Model
- Artifact Storage
- Audit
- Event Bus
- Job System
- Backup
- Security
- Observability

98. REST RESOURCE MODEL

The API may expose lifecycle resources under:

- /api/v1/projects/{project_id}/lifecycle
- /api/v1/projects/{project_id}/provenance
- /api/v1/projects/{project_id}/artifacts
- /api/v1/projects/{project_id}/exports

Administrative endpoints may be exposed separately.

99. FIRST AUTOMATED LIFECYCLE TEST

The first lifecycle integration test should execute:

1. Create project
2. Create model
3. Create Revision 1
4. Generate drawing
5. Generate document package
6. Hash document package
7. Record acceptance
8. Finalize revision
9. Verify provenance
10. Export project
11. Verify export manifest
12. Archive project
13. Verify archived artifacts
14. Verify historical revision reconstruction
15. Create retention candidate
16. Apply hold
17. Verify deletion is blocked
18. Release hold
19. Verify deletion eligibility
20. Execute controlled deletion
21. Record deletion event
22. Verify audit history

This becomes the first major regression test for the data lifecycle subsystem.

100. MVP DATA LIFECYCLE SUCCESS CRITERIA

The subsystem is successful when OneDraft can:

Identify ownership of project data

Classify persistent data

Track lifecycle states

Preserve historical model versions

Preserve revision provenance

Hash authoritative artifacts

Track source documents

Track AI-generated changes

Track human-generated changes

Preserve regulatory provenance

Preserve validation provenance

Preserve document provenance

Record customer acceptance provenance

Archive project data

Verify archived data

Apply retention policies

Create and enforce holds

Identify deletion candidates

Execute controlled deletion

Detect orphan artifacts

Verify provenance

Reconstruct historical project state

Export project data with a provenance manifest

101. SYSTEM INTEGRATION REQUIREMENTS

The Data Lifecycle, Retention & Provenance System integrates directly with:

Identity, Access Control & Security System
Runtime Orchestrator
Job & Worker Execution System
Artifact & Storage Management Engine
Integration & Event Bus
Observability, Telemetry & System Health
Backup, Disaster Recovery & Business Continuity
Project Lifecycle & State Machine
Compliance & Regulatory Engine
Validation & Verification Engine
Documentation & Drawing Generation Engine
Project Review, Redline, Acceptance & Approval Engine

No subsystem shall independently establish competing project provenance rules.

102. AUTHORITATIVE DATA HIERARCHY

The OneDraft authoritative hierarchy remains:

PROJECT MODEL
↓
MODEL VERSION
↓
REVISION
↓
REGULATORY MANIFEST
↓
VALIDATION
↓
DOCUMENTATION
↓
ACCEPTANCE
↓
AUDIT

The lifecycle subsystem governs preservation and traceability across this hierarchy.

103. DATA LIFECYCLE ARCHITECTURAL BOUNDARY

The subsystem shall not become a second project model.

Its responsibility is:

PRESERVE
CLASSIFY
TRACE
VERSION
RETAIN
ARCHIVE
VERIFY
DELETE

The Project Model remains responsible for the semantic state of the project.

104. FINAL ARCHITECTURAL DECISION

OneDraft shall treat project information as a **versioned, attributable, traceable and recoverable information system**.

The system shall not depend upon filenames, folder structures or individual database rows as the sole source of project provenance.

Instead:

PROJECT STATE
+
VERSION
+
REVISION
+
SOURCE
+
HASH
+
EVENT
+
ARTIFACT
+
ACCEPTANCE

collectively establish the historical identity of a project deliverable.

This provides the foundation required for reliable reconstruction, regulatory traceability, controlled archival, customer acceptance and long-term project integrity.

105. SPECIFICATION STATUS

ONEDRAFT DATA LIFECYCLE, RETENTION & PROVENANCE SYSTEM v0.1

STATUS: ESTABLISHED

The subsystem now defines:

Data ownership

Data classification

Lifecycle states

Retention policies

Historical state preservation

Artifact provenance

Model provenance

Regulatory provenance

Validation provenance

Document provenance

AI provenance

Human provenance

Hash integrity

Archival

Legal/administrative holds

Controlled deletion

Data lineage

Project reconstruction

Export provenance

Lifecycle auditing

Cross-subsystem lifecycle integration